



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{ 'wall_ahead': True/False, 'food_here': True/False }` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A mathematically perfect Table-Driven Agent is impossible for complex environments like Chess because the number of possible percept sequences becomes extremely large. A table-driven agent needs to store every possible situation and the correct action for each one. As the agent's lifetime increases, the number of possible percept histories grows very increasingly, causing the table size and required memory to become impossible to manage. Therefore, table-driven agents are not practical for complex environments.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The condition-action rules are represented by the IF statements inside the `sense_and_act()` method. Below can be stated as examples,

```
if percept["food_here"]:  
    return "Suck"
```

This is a rule where the condition is "food is present" and the action is "suck".
Another rule is:

```
if percept["wall_ahead"]:  
    return "Left"
```

This means if the agent detects a wall ahead, it performs the action of turning left.

The final rule:

```
return "Up"
```

Now, this acts as the default condition-action rule, where the agent moves forward when no other condition is true. These rules only depend on the current percept and do not use any previous information.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The `SimpleReflexAgent` got stuck because it only reacts to the current percept and does not remember previous states or actions. Since the environment is partially observable, the agent cannot see the complete map or know its previous locations. When it encounters the same situation again, it makes the same decision because it has no percept history or internal memory. For example, if the agent repeatedly detects a wall ahead, it will always execute the same rule, such as turning left. This can cause it to repeat the same movement cycle indefinitely because it cannot recognize that it has already visited that area before.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The ModelBasedAgent handles the Transition Model by updating its internal state after every action. The visited_cells set stores previously visited locations, and variables such as last_action and position track the agent's movement history. This allows the agent to recognize repeated paths and choose different actions. The Sensor Model is handled through the sense_and_act() method, where the agent receives percepts such as wall_ahead and food_here from the environment. The agent uses this information to update its internal state and decide the next action. Unlike the SimpleReflexAgent, the ModelBasedAgent combines current sensor information with previous memory to make better decisions.

Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING